

Object Detection Project

Full Presentation Script & What's Next

YOLO-Based Real-Time Object Detection · 3 Phases · 8 Steps

PHASE 1 — Data Collection & Preparation

| Step 1 Identifying the Right Object Classes

◆ SLIDE SCRIPT

Before collecting a single image, the most critical decision was what to detect. Not all object classes are equal — some are too rare, too ambiguous, or simply irrelevant to our goal. We focused strictly on objects that people commonly carry on a daily basis, ensuring they would appear realistically in a live camera frame. This targeted approach prevented wasted labeling effort and set a solid foundation for everything that followed.

◆ KEY SLIDE BULLETS

- Objects must be visually distinct and commonly carried daily
- Avoid classes too similar to each other — causes model confusion
- Each class must serve a real-world detection purpose
- Choosing well here defines the quality of all future steps

◆ PRESENTER NOTE



PRESENTER NOTE

Choosing the wrong classes at this stage is the most expensive mistake in a vision project — it can invalidate weeks of work. Every class must justify its existence.

| Step 2 Collecting Data from Multiple Sources

◆ SLIDE SCRIPT

A YOLO model learns by seeing thousands of examples of each object from different angles, lighting conditions, and backgrounds. A narrow dataset produces a fragile model. We gathered images from

Roboflow, Kaggle, and other open repositories — deliberately using multiple sources to maximize diversity in image quality, resolution, and scene context. This variety is what allows a model trained in a lab to still perform reliably in the real world.

◆ KEY SLIDE BULLETS

- Roboflow: structured datasets with existing annotations
- Kaggle: large community-uploaded image collections
- Additional open-source and domain-specific repositories
- Multi-source diversity leads to stronger model generalization

◆ PRESENTER NOTE



PRESENTER NOTE

A model trained on diverse data generalizes. A model trained on uniform data memorizes. The difference only becomes visible during live testing — often too late to fix easily.

| Step 3 Filtering, Labeling & Building the Workspace

◆ SLIDE SCRIPT

Raw data is never training-ready. We removed blurry, duplicate, and partially-visible images before accepting anything into our dataset. All accepted images were organized inside a shared Roboflow workspace for version control and team consistency. For every image sourced from Kaggle or other unannotated repositories, we drew bounding boxes manually — image by image — ensuring every label was tight, accurate, and uniform. Quality here directly determines model accuracy later.

◆ KEY SLIDE BULLETS

- Removed blurry, duplicate, and low-quality images first
- All images versioned inside a shared Roboflow workspace
- Bounding boxes drawn manually for all unannotated images
- Labels verified for tightness, accuracy, and consistency

◆ PRESENTER NOTE



PRESENTER NOTE

Manual labeling is the most time-consuming part of any vision project — but also the most impactful. One consistently mislabeled class can corrupt an entire detection category.

PHASE 2 — Model Training & Optimization

| Step 4 Selecting & Training the YOLO Model

◆ SLIDE SCRIPT

With a clean labeled dataset in place, we moved into model training. We chose the YOLO family for its exceptional balance between detection speed and accuracy, making it ideal for real-time applications. We evaluated multiple YOLO versions and selected the one most appropriate for our dataset size and class count. Training parameters — including image size, batch size, epochs, and confidence thresholds — were carefully configured and monitored throughout to ensure healthy convergence and avoid overfitting.

◆ KEY SLIDE BULLETS

- YOLO chosen for real-time speed and strong accuracy balance
- Evaluated multiple versions — selected best fit for our data scale
- Configured: image size, batch size, epochs, confidence threshold
- Monitored training curves to catch overfitting early

◆ PRESENTER NOTE



PRESENTER NOTE

The model architecture is just a tool. The real intelligence comes from the quality of data fed into it. A well-trained smaller model consistently outperforms a larger model on poor data.

| Step 5 Testing with Unseen Images

◆ SLIDE SCRIPT

After training completed, we moved into structured evaluation using a dedicated test set — images the model had never seen during training, carefully separated before the process began. We evaluated performance on a per-label basis, meaning each object class was assessed individually. This approach reveals which specific classes the model handles confidently and which require targeted improvement, rather than hiding weaknesses behind a high overall average score.

◆ KEY SLIDE BULLETS

- Strictly separated test set — never exposed during training
- Evaluated detection accuracy per label, not just overall mAP
- Measured precision, recall, and confidence score per class
- Identified underperforming classes as specific improvement targets

◆ PRESENTER NOTE

PRESENTER NOTE

An overall accuracy score can hide serious weaknesses in individual classes. Per-label evaluation is the only honest way to know where your model truly stands before deployment.

| Step 6 Iterating to Reach 89%+ Accuracy Per Label

◆ SLIDE SCRIPT

Achieving strong accuracy on the first run is rare. Model improvement is an iterative cycle: train, evaluate, identify weak classes, make changes, and repeat. Our benchmark was clear — every label needed to reach at least 89% accuracy. When a class fell below the threshold, we used two strategies: changing the model version or hyperparameters, or improving the data for that specific class by adding diverse images, removing ambiguous examples, or combining overlapping sub-classes.

◆ KEY SLIDE BULLETS

- Target: minimum 89% detection accuracy per individual class
- Strategy A — switch YOLO version or adjust hyperparameters
- Strategy B — add, remove, modify, or combine images per class
- All labels re-evaluated after every single iteration cycle

◆ PRESENTER NOTE

PRESENTER NOTE

Most accuracy gains in real projects come from better data, not a better model. When a class underperforms, always look at the data quality and quantity first.

PHASE 3 — Final Testing & Deployment

Step 7 Final Data Review & Quality Assurance

◆ SLIDE SCRIPT

Before the final training run, we conducted a full audit of the entire dataset. After many rounds of additions, deletions, and modifications, it is critical to confirm the dataset is internally consistent — no outdated labels, no conflicting class definitions, and balanced train/validation/test splits. We reviewed each class one final time and locked the definitive dataset version in Roboflow, ensuring the model trained from it was clean, complete, and production-ready.

◆ KEY SLIDE BULLETS

- Full audit of all labels across every class for consistency
- Verified train / validation / test split ratios are correct
- Removed residual noisy or incorrectly labeled images
- Locked final dataset version before the last training run

◆ PRESENTER NOTE



PRESENTER NOTE

Think of this as a pre-flight checklist. A small error overlooked here — one mislabeled class or an imbalanced split — can silently degrade the final model you will present.

Step 8 Live Camera Testing with OpenCV

◆ SLIDE SCRIPT

The final and most revealing test is not on a static dataset — it is in the real world, in real time. We integrated our trained YOLO model with OpenCV, processing each live camera frame and drawing bounding boxes, class labels, and confidence scores directly on the video stream. This test exposes how the model handles real lighting, motion blur, partial occlusions, and the natural unpredictability of live environments that no static dataset can fully replicate.

◆ KEY SLIDE BULLETS

- YOLO model integrated with OpenCV for real-time inference
- Live frames annotated with bounding boxes and confidence scores
- Tested across varied lighting, distances, angles, and backgrounds
- Final validation: real-world performance, not just test-set numbers

◆ PRESENTER NOTE



PRESENTER NOTE

If the model performs well on static images but struggles live, the cause is usually motion blur or insufficient training variety. This test reveals the truth about real-world readiness.

WHAT'S NEXT — Future Improvements

Our project does not end here — it marks the beginning of a larger vision. The current system successfully detects everyday objects in real time, but there is significant room to grow both in capability and in impact. The following five directions represent the most meaningful improvements we can pursue, each building directly on what we have already built and validated.

◆ QUICK OVERVIEW — 5 FUTURE IDEAS

- **Expanding the Class Library** — Add new object classes to broaden detection coverage
- **Public Testing via an Interactive GUI** — Launch a user-facing GUI for real people to try the system live
- **Structured User Feedback Collection** — Collect structured feedback through a built-in survey form after each session
- **Model Optimization for Edge Deployment** — Compress and optimize the model for mobile and embedded devices
- **Multi-Layer Object Identification** — Move beyond material recognition toward full object identity detection

01 Expanding the Class Library

| **Slide bullet:** *Add new object classes to broaden detection coverage*

◆ FULL SCRIPT

The objects we trained on were selected to prove the concept, but the real world contains far more variety. In future iterations, we plan to introduce additional classes — such as glasses, headphones, umbrellas, and other personal items — systematically going through the same rigorous collection, filtering, and labeling pipeline we established. Each new class must meet our 89% accuracy threshold before being considered production-ready. The stronger our class library, the more versatile and commercially applicable the system becomes.

02 Public Testing via an Interactive GUI

| **Slide bullet:** *Launch a user-facing GUI for real people to try the system live*

◆ FULL SCRIPT

One of the most honest forms of evaluation is putting your system in the hands of people who had nothing to do with building it. We plan to develop a clean graphical user interface that allows anyone — regardless of technical background — to interact with the model live through their own camera. This removes the controlled lab environment entirely and exposes the model to real diversity: different lighting setups, object orientations, and backgrounds we never anticipated during training.

03 Structured User Feedback Collection

Slide bullet: *Collect structured feedback through a built-in survey form after each session*

◆ FULL SCRIPT

Alongside the GUI, we will integrate a simple feedback form that appears after each testing session. Users will be able to rate detection accuracy, flag missed objects, and suggest classes they expected the system to recognize but did not. This transforms every user interaction into a data point for improvement. Rather than guessing what to fix next, we will have direct evidence of where the model falls short in real-world use — a feedback loop that makes every version smarter than the last.

04 Model Optimization for Edge Deployment

Slide bullet: *Compress and optimize the model for mobile and embedded devices*

◆ FULL SCRIPT

Currently the model runs on a standard computer with a connected camera. A natural next step is to compress and optimize it for deployment on edge devices — such as Raspberry Pi, mobile phones, or embedded security modules — where processing power is limited. Techniques like model quantization and pruning can drastically reduce model size while preserving most of the accuracy. This opens the door to real-world applications in smart security checkpoints, inventory tracking, and portable detection systems.

05 Multi-Layer Object Identification

Slide bullet: *Move beyond material recognition toward full object identity detection*

◆ FULL SCRIPT

Current automated detection systems often stop at recognizing what category an object belongs to. But knowing that something is a "metal object" is not enough — a coin and a key are both metal, yet they require completely different handling in any intelligent system. Our vision for the next phase is a multi-layer identification model that recognizes not just the substance or category, but the specific identity of the object. This deeper level of understanding is what separates a demonstration project from a truly intelligent detection system with real industrial and logistics value.

◆ PRESENTER NOTE — WHAT'S NEXT SLIDE



PRESENTER NOTE

When presenting this slide, frame it as ambition grounded in proof. You have already done the hard part — building a working, accurate system from scratch. Everything here is the natural extension of that work, not speculation. The feedback form idea in particular tends to resonate strongly with audiences because it shows you understand that real-world performance matters more than lab results.